"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING

Faculty of computer science and engineering

JavaScript Functions, Arrays and the DOM

Advanced Web Design

Variable declarations: var, let and const

	Scope	Reassignable	Mutable
const	block	No	Yes
let	block	Yes	Yes
var	function	Yes	Yes

Here, mutable refers to object and array contents. Primitive values are immutable.

Functions

- **Function statements (1):** Declared with the function keyword and can be called before it's defined.

- **Function expressions (2):** A function stored in a variable, must be defined before use.

Functions

(1)

```
sayWelcome(); // works — hoisted

function sayWelcome() {
    console.log("Welcome to AWD!");
}
```

(2)

```
// greet(); ReferenceError
let greet = function sayWelcome() {
    console.log("Welcome to AWD!");
}
greet(); // works
```

References and values in JS

- Every primitive type is passed by value (copied).
- Object references are also copied by value.

References and values in JS

- Pass by value:

```
let num1 = 10;

function changeValue(value) {
    value = 20;       // change the local copy
    console.log("Inside function:", value); // value = 20
}

changeValue(num1);
console.log("Outside function:", num1); // value = 10
```

References and values in JS

- Object reference:

```
let student = {
    name: "John",
    idNum: 203043
};

function changeName(student) {
    student.name = "Jane"
    console.log("Inside function:", student)
}

changeName(student);
console.log("Outside function:", student)
```

Functions as arguments

- In JavaScript, functions are “first-class citizens”
- They can be assigned to variables or passed to other functions as arguments
- Callbacks are common in array methods and DOM events.

Functions as arguments

- Example

```
function sayHello(){
    console.log("Hello students, welcome!")
}

function greetStudents(someFunction) {
    console.log("Preparing to greet the students...")
    someFunction()
    console.log("How are you?")
}

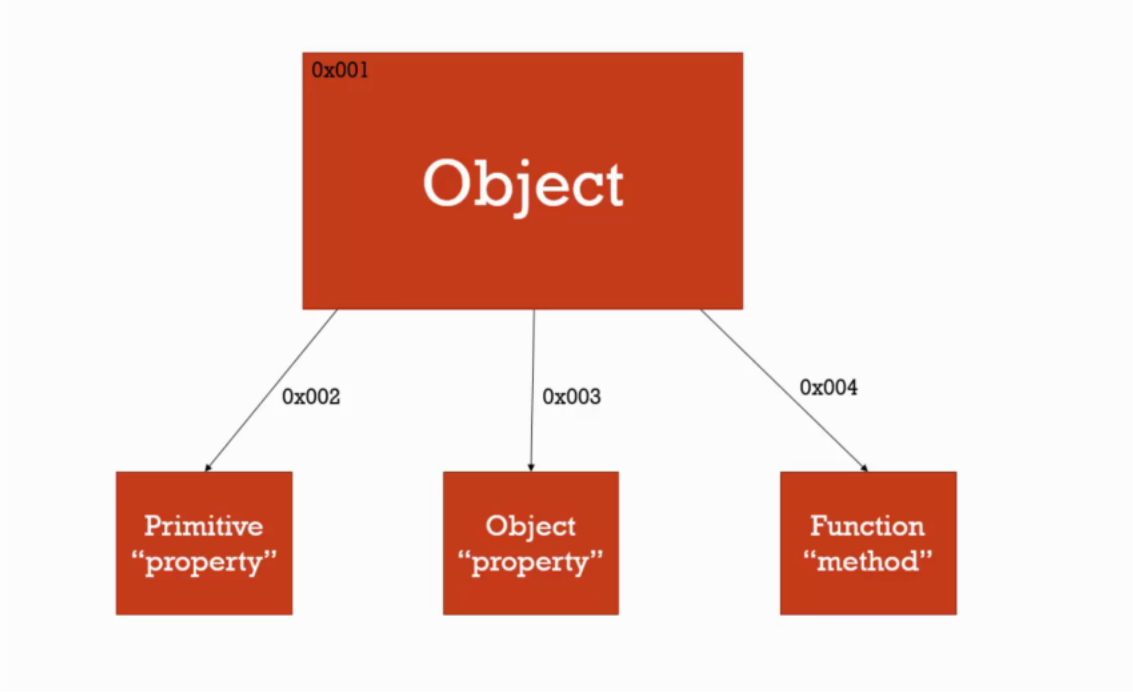
greetStudents(sayHello)
```

Objects in depth

- Collections of key-value pairs
- The value portion can be any type of value:
  - number
  - string
  - array
  - function
  - another object

Objects in depth

- Objects can store primitive values and references to other objects.



Closures

- A **closure** happens when a function **remembers** variables from the place it was **created**.

- Inner functions can still **access outer variables**, even after the outer function is finished.

Closures

- Example

```
function outer(){
    let faculty = "FINKI"
    function inner(){
        console.log(`Hello students! Welcome to ${faculty}!`);
    }
    return inner;
}

let fn = outer();
fn();
```

Closures

- Write a function makeCounter that returns another function. Each time the returned function runs, increase its own count by 1 and return the new value. Start at 0 and demonstrate that two counters keep separate counts.

```
const first = makeCounter();
const second = makeCounter();

console.log(first());  // 1
console.log(first());  // 2
console.log(second()); // 1
```

Solution

- The returned function remembers the count from makeCounter.

```
function makeCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const first = makeCounter();
const second = makeCounter();
console.log(first());  // 1
console.log(first());  // 2
console.log(second()); // 1
```

Useful array functions

Method & Description
concat()Returns a new array comprised of this array joined with other array(s) and/or value(s).
every()Returns true if every element in this array satisfies the provided testing function.
filter()Creates a new array with all of the elements of this array for which the provided filtering function returns true.
forEach()Calls a function for each element in the array.
indexOf()Returns the first (least) index of an element within the array equal to the specified value, or -1 if none is found.
join()Joins all elements of an array into a string.
lastIndexOf()Returns the last (greatest) index of an element within the array equal to the specified value, or -1 if none is found.
map()Creates a new array with the results of calling a provided function on every element in this array.
pop()Removes the last element from an array and returns that element.
push()Adds one or more elements to the end of an array and returns the new length of the array.
reduce()Apply a function simultaneously against two values of the array (from left-to-right) as to reduce it to a single value.
reduceRight()Apply a function simultaneously against two values of the array (from right-to-left) as to reduce it to a single value.
reverse()Reverses the order of the elements of an array -- the first becomes the last, and the last becomes the first.
shift()Removes the first element from an array and returns that element.
slice()Extracts a section of an array and returns a new array.
some()Returns true if at least one element in this array satisfies the provided testing function.
toSource()Represents the source code of an object
sort()Sorts the elements of an array
splice()Adds and/or removes elements from an array.
toString()Returns a string representing the array and its elements.
unshift()Adds one or more elements to the front of an array and returns the new length of the array.

Testing type with *typeof*

- **typeof** checks what type of value a variable holds.
- Always returns a **string** describing the type.
- Works on numbers, strings, booleans, objects, and more.
- Great for quick checks in conditionals or debugging.

Arrow functions

- A shorter way to write function using the => syntax
- Automatically return the expression if written in one line.
- Great for callbacks and inline logic

Comparison: Regular function vs Arrow function

(1)

```
// Regular function
function add(a, b) {
    return a + b;
}


// Arrow function
const addArrow = (a, b) => a + b;
```

(2)

```
// Regular function
function isEven(num) {
    return num % 2 === 0;
}


// Arrow function
const isEvenArrow = num => num % 2 === 0;
```

filter() and map()

- filter() keeps matching elements.
- map() creates one new value for each element.

```
const numbers = [1, 2, 3, 4, 5];
const even = numbers.filter(number => number % 2 === 0);
const doubled = even.map(number => number * 2);

console.log(even);    // [2, 4]
console.log(doubled); // [4, 8]
console.log(numbers); // [1, 2, 3, 4, 5]
```

forEach() and reduce()

- forEach() executes a callback and returns undefined.
- reduce() combines elements into an accumulated result.

```
const quantities = [2, 1, 3];

quantities.forEach(quantity => console.log(quantity));

const total = quantities.reduce(
  (sum, quantity) => sum + quantity, 0
);
console.log(total); // 6
console.log([].reduce((sum, value) => sum + value, 0));
// 0
```

Sorting arrays

- sort() changes the array it is called on.
- Copy the array first to keep the original order.
- Use a comparator when sorting numbers.

```
const prices = [30, 8, 120, 15];
const ascending = [...prices].sort((a, b) => a - b);
const descending = [...prices].sort((a, b) => b - a);

console.log(ascending);  // [8, 15, 30, 120]
console.log(descending); // [120, 30, 15, 8]
console.log(prices);     // [30, 8, 120, 15]
```

Example 2

- Create a script that filters out products that are out of stock and uses array functions to:
  - list the names of the available items
  - calculate the total quantity of all available items

```
[{ id: 1, name: "Keyboard", price: 30, qty: 2, inStock: true },
 { id: 2, name: "Mouse", price: 15, qty: 1, inStock: false },
 { id: 3, name: "Monitor", price: 120, qty: 1, inStock: true },
 { id: 4, name: "Headphones", price: 45, qty: 3, inStock: true },
 { id: 5, name: "USB Cable", price: 8, qty: 5, inStock: false }]
```

Solution

```
let products = [
  { id: 1, name: "Keyboard", price: 30, qty: 2, inStock: true },
  { id: 2, name: "Mouse", price: 15, qty: 1, inStock: false },
  { id: 3, name: "Monitor", price: 120, qty: 1, inStock: true },
  { id: 4, name: "Headphones", price: 45, qty: 3, inStock: true },
  { id: 5, name: "USB Cable", price: 8, qty: 5, inStock: false }
]

let filteredProducts = products.filter(product => product.inStock)
let filteredProductsNames = filteredProducts.map(product => product.name)
let totalQuantity = 0
filteredProducts.forEach(product => totalQuantity += product.qty)

console.log(filteredProductsNames)
console.log("Total quantity: ", totalQuantity)
```

Example 3

- For the given an array of user objects, write a function that removes duplicate users while keeping only the first occurrence of each email. After cleaning the array, extract only the users' names, sort them alphabetically, and print the result as a single comma-separated string.

```
[
  { id: 1, name: "Ana", email: "ana@example.com" },
  { id: 2, name: "Marko", email: "marko@example.com" },
  { id: 3, name: "Ana", email: "ana@example.com" },
  { id: 4, name: "Mila", email: "mila@example.com" },
  { id: 5, name: "John", email: "john@example.com" },
  { id: 6, name: "Jane", email: "jane@example.com" }
]
```

Solution

```
const users = [
    { id: 1, name: "Ana", email: "ana@example.com" },
    { id: 2, name: "Marko", email: "marko@example.com" },
    { id: 3, name: "Ana", email: "ana@example.com" },
    { id: 4, name: "Mila", email: "mila@example.com" },
    { id: 5, name: "John", email: "john@example.com" },
    { id: 6, name: "Jane", email: "jane@example.com" }
];

const emails = []
const filteredUsers = []

users.forEach(user => {
    if(!emails.includes(user.email)){
        filteredUsers.push(user);
    }
    emails.push(user.email)
})

let filteredUsersNames = filteredUsers
    .map(user => user.name)
    .sort()
    .join(",")

console.log(filteredUsersNames)
```

Example 4

- Compute total grade sum for students above 10 by composing map, filter and reduce:

```
const students = [
   { name: "Nick", grade: 10 },
   { name: "John", grade: 15 },
   { name: "Julia", grade: 19 },
   { name: "Nathalie", grade: 9 },
];
```

Solution

```
const students = [
    { name: "Nick", grade: 10 },
    { name: "John", grade: 15 },
    { name: "Julia", grade: 19 },
    { name: "Nathalie", grade: 9 },
];

let totalGrade = students
    .filter(student => student.grade > 10)
    .map(student => student.grade)
    .reduce((prev, next) => prev+next, 0)

console.log(totalGrade)
```

Destructuring objects and arrays

- *Destructuring* is a convenient way of creating new variables by extracting some values from data stored in objects or arrays.

- To name a few use cases, *destructuring* can be used to destructure function parameters or *this.props* in React projects for instance.

Destructuring objects and arrays

```
const person = {
  firstName: "Nick",
  lastName: "Anderson",
  age: 35,
  sex: "M"
}
```

- Without destructuring:

```
const first = person.firstName;
const age = person.age;
const city = person.city || "Paris";
```

- With destructuring, all in one line:

```
const { firstName: first, age, city = "Paris" } = person;
```

Spread operator "..."

- The spread operator ... is used to expand elements of an iterable (like an array) into places where multiple elements can fit

```
const arr1 = ["a", "b", "c"];
const arr2 = [...arr1, "d", "e", "f"]; // ["a", "b", "c", "d", "e", "f"]
```

Example 5

- Given the array of orders, write a script that processes only paid orders. Calculate their total revenue, group the revenue by customer and sort the customer totals from highest to lowest. Print each customer as Name - total and check whether any paid order is worth at least 50. Use array methods and keep the original array unchanged.

```
const orders = [
  { id: 1, customer: "Ana", total: 40, status: "paid" },
  { id: 2, customer: "Marko", total: 25, status: "pending" },
  { id: 3, customer: "Ana", total: 30, status: "paid" },
  { id: 4, customer: "Mila", total: 60, status: "paid" },
  { id: 5, customer: "Marko", total: 15, status: "paid" }
];
```

Solution

- Example 5: filter paid orders and calculate the total revenue.
- Reuse the orders array from the previous slide.

```
const paidOrders = orders.filter(order => order.status === "paid");
const revenue = paidOrders.reduce(
  (sum, order) => sum + order.total, 0
);
const hasLargeOrder = paidOrders.some(order => order.total >= 50);

console.log(revenue);       // 145
console.log(hasLargeOrder); // true
console.log(orders.length); // 5, the original array is unchanged
```

Solution

- Example 5: use an object accumulator to group totals by customer.

```
const revenueByCustomer = paidOrders.reduce((totals, order) => {
  totals[order.customer] = (totals[order.customer] ?? 0) + order.total;
  return totals;
}, {});

const customerTotals = Object.entries(revenueByCustomer)
  .map(([name, total]) => ({ name, total }))
  .sort((a, b) => b.total - a.total);

customerTotals
  .map(({ name, total }) => `${name} - ${total.toFixed(2)}`)
  .forEach(line => console.log(line));
// Ana - 70.00, Mila - 60.00, Marko - 15.00
```

Data processing and the DOM

- Keep the data in an array. Use array methods to prepare the values you want to display.
- Select page elements with querySelector() and create list items with createElement().
- Use textContent to display text. Use addEventListener() to update the view after user input.

Example 6

- Create a page that displays the products from Example 2 and allows the user to search by name and sort by price. The search should ignore letter case and surrounding spaces. Use array functions to filter and sort the available products without changing the original array, then update the DOM to show their names, prices and quantities. Display the number of results and their total quantity. If no products match, display "No products found".

Solution

- Example 6: HTML structure. Place the script after these elements.
- Reuse the complete products array from Example 2.

```
<label>Search <input id="search" type="search"></label>
<label>Price order
  <select id="order">
    <option value="asc">Lowest first</option>
    <option value="desc">Highest first</option>
  </select>
</label>
<p id="summary" aria-live="polite"></p>
<ul id="products"></ul>
```

Solution

- Example 6: selecting and transforming the data.

```
const search = document.querySelector("#search");
const order = document.querySelector("#order");
const list = document.querySelector("#products");
const summary = document.querySelector("#summary");

function visibleProducts() {
  const query = search.value.trim().toLowerCase();
  return products
    .filter(product => product.inStock &&
      product.name.toLowerCase().includes(query))
    .sort((a, b) => order.value === "asc"
      ? a.price - b.price : b.price - a.price);
}
```

Solution

- Example 6: rendering the result and handling input events.

```
function updateView() {
  const visible = visibleProducts();
  list.replaceChildren();
  visible.forEach(product => {
    const item = document.createElement("li");
    item.textContent =
      `${product.name} - ${product.price} - Qty: ${product.qty}`;
    list.appendChild(item);
  });
  if (!visible.length) list.textContent = "No products found";
  const quantity = visible.reduce((sum, p) => sum + p.qty, 0);
  summary.textContent =
    `${visible.length} products. Total quantity: ${quantity}`;
}
search.addEventListener("input", updateView);
order.addEventListener("change", updateView);
updateView();
```